ESTR1002B Tutorial 1

Hoover H.F. Yin

Department of Information Engineering
The Chinese University of Hong Kong

January 18, 2017

The limit of a computer

Contact

- YIN Ho Fai, Hoover
- Office: Institute of Network Coding (INC), 7/F, SHB
- Email: yhf015 at ie dot cuhk dot edu dot hk

- Can speak Cantonese

Outline

1. Why programming?
2. C compiler
3. `int`
4. `float` vs. `double`

The example codes in this tutorial are uploaded to eLearn

Historical Notes

The design of C was based on another language called B.
C was named because it is "after B".

Why Programming?

Computer is stupid, but loyal

- Stupid means:
  - It can't think
  - It can only do simple operations (e.g. +-*/)
  - It can only follow instructions sequentially
- Loyal means:
  - It won't question your instructions
  - It follows your instructions exactly
  - If there's a bug, it must be your problem

You need to write instructions on how to solve your problem!
How? Programming

Why Programming?

- Computers can't understand your C code!
- Why do we write in C !!!!????
  - Human readable
  - Better view to program flows
  - Can perform low-level jobs like controlling the memory
- What can a computer understand? Machine Language
- How to let a computer understand your C code? Compile it!

What machine language looks like?

01001010001010010010110101001010 10

I think you don't want to code in this way...

C Compiler

There are different C compilers in the world...

- CL (Microsoft C compiler, which is used by Visual Studio)
- GCC (the one used by the online compiler)
- LLVM (a modified LLVM is used by Xcode)
- BCC (Borland C compiler)
- ... and more!

In this tutorial, the output of the examples are given by GCC.

Historical Notes

In the very old days, the full name of GCC is GNU C Compiler.
Nowadays, the full name of GCC is GNU Compiler Collection.
What is GNU? Search "GNU Project" on the Internet!

C Compiler

Contains 4 major components:

1. Preprocessor
   - Process `#include`, etc.
   - Output a processed source file
2. Compiler
   - Turn the processed code into assembly code
   - (optional) Optimize your code
3. Assembler
   - Translate the assembly code into machine code
   - The output is called object file
4. Linker
   - Combine object files into an executable

Preprocessor

Most lines start with # are for the preprocessor

Example of #include:

preprocess.c

```
#include "preprocess.h"

int main(void)
{
    //Hello :D
    int hi = 0;
    return 0;
}
```

preprocess.h

```
int i;
float j;
double k;
```

#include

Why it is `#include "preprocess.h"` but not `#include <preprocess.h>`

- `"preprocess.h"` means the preprocessor will search the file `preprocess.h` in the current directory first, then the default directory
- `<preprocessor.h>` means the preprocessor will only search the file in the default directory
- We used to use `" "` for custom header files

Why the extension is `.h`

- `h` stands for `header`
- The files with extension `.h` are called header files
- It is called header because... they used to be included at the header (but not necessary) of your source code!

Yes, `stdio.h` is a file, and it is in the "default directory". Try to find out where `stdio.h` is in your computer!

Preprocessor

Now we want to get the output of the preprocessor (using GCC)

Compilation Command

```
gcc -E preprocess.c -o preprocess.i
```

- `-E` means only run the preprocessor
- `-o preprocess.i` means write the output in a file named `preprocess.i`
- `.i` is the extension for preprocessed files
- The full command means: Run preprocessor on the file named `preprocess.c` and output the result in another file named `preprocess.i`

Preprocessor

We can have similar command if you use CL in cmd

Compilation Command

```
cl /P preprocess.c
```

Or, configure it in the IDE of Visual Studio. Read
https://msdn.microsoft.com/en-us/library/8z9z0bx6.aspx
for more details.

We will skip the compilation commands of all compilers except GCC in this tutorial.
You may find more compiler options of CL in
https://msdn.microsoft.com/en-us/library/fwkeyyhe.aspx

#include

The output of the preprocessor:

preprocess.i

```
# 1 "preprocess.c"
# 1 "<built-in>"
# 1 "<command-line>"
# 1 "preprocess.c"
# 1 "preprocess.h" 1
int i;
float j;
double k;
# 2 "preprocess.c" 2
```

preprocess.i (cont.)

```
int main(void)
{

 int hi = 0;
 return 0;
}
```

#include

We still have some lines start with #

- They are linemarkers. Can be surpressed by using -P in GCC
- We can treat the linemarkers as comments for you

What did the preprocessor do here?

- Copy and paste the #include files into your code!
- Comments in C code are deleted, but not necessary
- Tabs are substituted into spaces, but not necessary

Historical Notes

Old style preprocessor won't do the "not necessary" part. Try the command

```
gcc -E preprocess.c -o preprocess_trad.i -traditional
```

In fact, we can pass non-C code to the C preprocessor!

printf and stdio.h

Why `#include <stdio.h>` if we use `printf`?

helloworld.c

```
#include <stdio.h>

int main(void)
{
    printf("Hello World!\n");
    return 0;
}
```

Have you tried to compile it without `#include <stdio.h>`?
You will get a warning!

Warning Message

```
incompatible implicit declaration of built-in function 'printf'
```

printf and stdio.h

What is that warning?

- The compiler doesn't know if you want to use `printf` in a way consistent to the built-in way
- For example, do we need to cast `"Hello World!\n"` to other data type?

To know what `#include <stdio.h>` is doing, we can check the preprocessor output

Compilation Command

```
gcc -E helloworld.c -o helloworld.i
```

Note: `stdio` stands for standard I/O (I/O means input/output), so don't misspell it as `studio`!!!

printf and stdio.h

The whole content of the file `stdio.h` is copied in `helloworld.i` and there are more than 800 lines! What we want is the line

helloworld.i (partial)

```
extern int printf (__const char *__restrict __format, ...);
```

What we want is the function prototype of `printf`!

- The function prototype of `printf` indicates how to use `printf` correctly
- It is totally okay if you can't understand this line at this moment
- We will come back to this a few weeks later

Note: `printf` stands for print formatted

#define

Except copy and paste, what else a preprocessor can do?

- Search and replace
- Conditional checking, etc. (not this time)

define.c

```
#include <stdio.h>

#define HI 2+3

int main(void)
{
    printf("%d\n", HI*HI);
    return 0;
}
```

#define

`#define` is simply search and replace

- Every `HI` in the program is replaced by 2+3
- It won't replace a part of a word, e.g., the text `HIHI` won't be replaced
- Case sensitive

So what is the output? We first compile the code

Compilation Command

```
gcc define.c -o define
```

- It means compile `define.c` into an executable named `define`
- * Please name it `define.exe` if you are using Windows
  - I won't mention it in the later slides
- `.exe` means executable. It is not necessary in Unix, Linux, Mac OS.

#define

Execution Command

```
./define
```

- In Windows, it is not necessary to type `./` before the program name

Output

```
11
```

Why it's not 25? Because it's search and replace!
Let's check what is the preprocessor output!

Compilation Command

```
gcc -E define.c -o define.i
```

#define

define.i (partial)

```
printf("%d\n", 2+3*2+3);
```

We do multiplication before addition, that's why.

Therefore we usually use a lot of parentheses in `#define` to get our expected output, e.g.

```
#define HI (2+3)
```

We will get 25 as the output because it becomes `(2+3)*(2+3)` after search and replace.

Compiler

Compiler?

- Turn the output of the preprocessor into assembly code
- (optional) Optimize your code

Assembly Language?

- Basically every line corresponds to one instruction (machine code)
  - There are some exceptions called pseudoinstructions, which corresponds to another instruction or more than one instructions
- Machine dependent!
  - Different between CPU architectures, like Intel and ARM
  - Different styles even for the same architecture, like Intel syntax and GAS syntax
  - You will know more if you take ELEG3230 / CENG3420 / CSCI3420 in the future

Assembly Language

Remember the `helloworld.i` we made by the preprocessor? Let's pass it to the compiler!

Compilation Command

```
gcc -S helloworld.i -o helloworld.s
```

- `-S` means output the assembly code
  - pass all `.c` files through preprocessor and compiler
  - pass all `.i` files through compiler only
- `.s` is the extension for assembly codes

The assembly file made by GCC is in GAS syntax. The output is also uploaded, but we are not going to read it at this moment.

Optimization

We can ask the compiler to optimize our code.

opt.c

```
#include <stdio.h>

int main(void)
{
    int a = 2;
    a += 2;
    a += 2;
    a += 2;
    a += 2;
    printf("%d\n", a);
    return 0;
}
```

Optimization

We compare the assembly codes...

Compilation Command

```
gcc -O0 -S opt.c -o opt0.s
gcc -O1 -S opt.c -o opt1.s
```

- -O0 means no optimization
- -O1 means level 1 optimization
- we also have -O2 and -O3 for further performance optimization
- and we have -Os for size optimization

Optimization

With -O0, our C code is translated directly...

opt0.s (partial)

```
movl    $2, -4(%rbp)
addl    $2, -4(%rbp)
addl    $2, -4(%rbp)
addl    $2, -4(%rbp)
addl    $2, -4(%rbp)
```

opt.c (partial)

```
int a = 2;
a += 2;
a += 2;
a += 2;
a += 2;
```

After that we still have

opt0.s (partial)

```
movl    $.LC0, %eax
movl    -4(%rbp), %edx
movl    %edx, %esi
movq    %rax, %rdi
movl    call printf
```

These are something like moving `"%d\n"` and the value of the variable a somewhere, then call `printf`

Optimization

With -O1...

opt1.s (partial)

```
    movl    $10, %esi
    movl    $.LC0, %edi
    call    printf
```

It simply means moving 10 and "%d\n" somewhere, then call printf, i.e., it becomes

```
printf("%d\n", 10);
```

Although computer calculates very fast, but remember, every instruction takes time! You will know more in CSCI2100 / ESTR2102 / CENG3420 / CSCI3420.

Assembler

Assembler translates the assembly code into machine code.
The output of the assembler is called `object file`.

Compilation Command

```
gcc -c opt.c -o opt.o
```

- `-c` means output the object file
  - similarly, we can pass in `.i` and `.s` files to skip the corresponding components
- `.o` is the extension for object files

We can create the object files even there is no `main()` in the source code.

The output is not human friendly now. You can't use a text editor to view it as there are unprintable characters.

xxd

If you still want to have a look (in a binary form), you can use the following command in Linux / Unix / Mac

Command

```
xxd opt.o
```

- xxd is a program to make a hexdump (print it as hexadecimal numbers but not 0 and 1 only)

Linker

Linker is the final component. It groups object files together and output an executable.

- Did you implement `printf`? No, of course
- Someone else implemented it, and the linker is now grabbing the code and other essentials to make your program runnable

Compilation Command

```
gcc opt.o
```

- By default, the linker will include the C library (or its reference)
  - Some older compilers won't include the math library
- The default name of the output by GCC is `a.out`
  - You can change the output name by using `-o`
- The executable is not uploaded, as it can't run in a computer with different CPU architecture and OS
  - Say, you can't run the executable for Windows in Mac OS

int

We need to give a meaning to a binary string. Otherwise the string is meaningless!

01010100101010001010010100111010

Each binary digit uses a bit. A few bits can only represent a small number of combinations.

- Therefore we used to use a group of bits: 8 bits = 1 byte

  01010100 10101000 10100101 00111010

- `int` by default in a 32-bit or 64-bit environment uses 4 bytes
  - `int` in a 16-bit environment uses 2 bytes
- We can have 2^{32} combinations using 4 bytes

For a non-negative integer, we can use the binary representation. Then how to represent a negative number?

2's Complement

Assumptions: We use N bits to store a number

- A positive number is represented by its binary base form
- A positive number plus its "negative number" equals 2^N

We use a smaller case here... $N = 4$

$$\begin{array}{cccccc} & & 0 & 0 & 1 & 1 \\ & + & 1 & 1 & 0 & 1 \\ \hline & 1 & 0 & 0 & 0 & 0 \end{array}$$

- 0011_2 is 3
- $0011_2 + 1101_2 = 2^4$
- So 1101_2 is -3 in 2's complement when $N = 4$

2's Complement

If we have a number x, how to calculate $-x$ by ourselves?

- By definition, $2^N - x$ is $-x$
- Or, we can use the trick below

Consider the previous example, $x = 0011_2$

- Let y be x after flipping all bits (0 to 1, 1 to 0), i.e. $y = 1100_2$
- What is $x + y$? $x + y = 1111_2$
- So $x + y + 1 = 2^N$, which implies $y + 1$ is $-x$

2's Complement

Why 2's complement?

- $3 = 0011_2$
- $-1 = 1111_2$

$$\begin{array}{cccccc} & 0 & 0 & 1 & 1 \\ + & 1 & 1 & 1 & 1 \\ \hline 1 & 0 & 0 & 1 & 0 \end{array}$$

- As we can only store N bits, the most significant bit is lost!
- So we have $3 + (-1) = 0010_2 = 2$

This is friendly to hardware design as we don't need an extra component to handle addition of a negative number

- More components mean more power (use more electricity)

2's Complement

What number is 1000_2?

$$\begin{array}{cccccc} & & 1 & 0 & 0 & 0 \\ & + & 1 & 0 & 0 & 0 \\ \hline & 1 & 0 & 0 & 0 & 0 \end{array}$$

Except zero and this case, all numbers with the most significant bit 1 are negative numbers!!!

- The bit is called the sign bit
- To be consistent, 1000_2 is a negative number, i.e., -8

2's complement is used in most widely used CPU architectures. Still there exists other number systems like

- 1's complement ($x + (-x) = 2^N - 1$, i.e. all N bits are 1)
- Sign magnitude representation (sign bit then the binary)

Note: There is no negative zero in 2's complement

int

What is the range of `int`?

- $[-2^{31}, 2^{31} - 1] = [-2147483648, 2147483647]$

What if we add 1 to $2^{31} - 1$?

$$\begin{array}{cccccc} & 0 & 1 & \dots & 1 & 1 \\ + & 0 & 0 & \dots & 0 & 1 \\ \hline 0 & 1 & 0 & \dots & 0 & 0 \end{array}$$

It becomes -2147483648. This is called overflow

Overflow

We can verify it with the following program

overflow.c

```
#include <stdio.h>

int main(void)
{
    int a = 2147483647;
    printf("%d\n", a+1);
    return 0;
}
```

Output

```
-2147483648
```

Overflow

You can think it as 'wrap around'

- When you minus 1 from -2^{31}, you will get $2^{31} - 1$
  - It is still called overflow
- Similar idea for other data types like `short`, `char`, `long`, and their `unsigned` forms

A good programmer should always aware of the limit of each data type!

Sometimes it's not harmful to have overflow, and even we can make use of it, for example

- Swap integers `x` and `y` by `x = x+y; y = x-y; x = x-y;`
- Rabin-Karp rolling hash implemented for file deduplication in cloud computing (CSCI4180)

float vs. double

Differences between float and double?

- float uses 4 bytes, double uses 8 bytes
- double can store a more accurate number

Why called float?

- Floating point number
- The idea is, we first store an integer
- Then we store where to put the radix point (decimal point)
- The point is floating, we can move it to other places
- A mathematical representation: scientific notation

Why called double?

- Double precision floating point number

Binary Number

We do not consider negative numbers here, as in maths we can use a negative sign.

How to convert a binary number to a decimal number?

$$(\ldots b_2 b_1 b_0 . b_{-1} b_{-2} \ldots)_2 = \sum_i b_i 2^i$$

For example, we have 101.101_2, then

$$1 \times 2^2 + 0 \times 2^1 + 1 \times 2^0 + 1 \times 2^{-1} + 0 \times 2^{-2} + 1 \times 2^{-3} = 5.625$$

How to convert a decimal number to a binary number?

- Convert integer and fractional parts individually

Binary Number

For the integer part, let $x = \sum_{i \geq 0} b_i 2^i$

- If b_0 is 0, it means x is an even number
- If b_0 is 1, then it is an odd number
- So b_0 equals the remainder of $x \div 2$
- Now we have $\lfloor x/2 \rfloor = \sum_{i \geq 1} b_i 2^{i-1}$
- Similarly we can get b_1, and so on

Take 5 as an example

1. $5 \div 2 = 2 \cdots 1$, so $b_0 = 1$
2. $2 \div 2 = 1 \cdots 0$, so $b_1 = 0$
3. $1 \div 2 = 0 \cdots 1$, so $b_2 = 1$
4. $0 \div 2 = 0$ and so on, so we know $b_3 = b_4 = \ldots = 0$

So we have $5 = (\ldots b_2 b_1 b_0)_2 = 101_2$

Binary Number

For the fractional part, let $y = \sum_{i<0} b_i 2^i$

- Consider $2y$
- If b_{-1} is 1, then $2 > 2y \geq 1$
- If b_{-1} is 0, then $0 < 2y < 1$
- So b_{-1} equals $\lfloor 2y \rfloor$
- Removing the integer part, we have $2y - \lfloor 2y \rfloor = \sum_{i<-1} b_i 2^{i+1}$
- Similarly we can get b_{-2} and so on

Take 0.625 as an example

1. $0.625 \times 2 = 1.25$, so $b_{-1} = 1$
2. $0.25 \times 2 = 0.5$, so $b_{-2} = 0$
3. $0.5 \times 2 = 1.0$, so $b_{-3} = 1$
4. $0 \times 2 = 0.0$ and so on, so we know $b_{-4} = b_{-5} = \ldots = 0$

So we have $0.625 = (0.b_{-1}b_{-2}\ldots)_2 = 0.101_2$

Floating Point Numbers

Scientific notation: $m \times 2^n$

- m is called the mantissa
- n is called the exponent

For example, $1.11_2 \times 2^1$ and $11.1_2 \times 2^0$ are representing the same number. We can define a normalized form such that the expression is unique (except 0).

Normalized form: $(1.b_1 b_2 \ldots)_2 \times 2^n$

Again, there are different representation of floating point numbers in computer. The most widely used CPU architectures nowadays use a standard called IEEE 754.

- In the normalized form, the first digit must be 1 so we don't need to store it, which is called the hidden bit or implicit bit
- The storage is divided into 3 parts: sign bit, exponent bits, and mantissa bits

IEEE 754

IEEE 754 fixed the number of bits in each part

- `float`: 1 sign bit, 8 exponent bits, 23 mantissa bits
- `double`: 1 sign bit, 11 exponent bits, 52 mantissa bits

How many significant digits in decimal?

- It is only related to the mantissa part
- Remember there is a hidden bit
- `float`: $\log_{10} 2^{24} \approx 7.2$, so there are 7 significant digits
  - That's why `%f` in `printf` by default has 1 integer digit and 6 decimal digits
- `double`: $\log_{10} 2^{53} \approx 15.9$, so there are 15 significant digits
  - In fact the precision is more than double

IEEE 754

How to store a negative exponent?

- In fact the exponent part is stored as a non-negative number that has a fixed bias
- The exponent part stores the value of the exponent plus the bias
- Bias of `float`: 127
- Bias of `double`: 1023

Take `float` as an example

- If the exponent is 0, we will store $0 + 127 = 127 = 01111111_2$
- If the exponent is -10, we will store $-10 + 127 = 117 = 01110101_2$

Note: Zero is a special case. We store all zero bits in both exponent and mantissa.

Floating Point Numbers

Now we know that

- The values floating point numbers can represent are discret and have a finite combinations
- In most cases we cannot store a real number exactly
- If a number is too small (close to 0), it will become 0
  - This is called underflow

There still have some special IEEE 754 representations like NaN (not a number), infinity, etc. You will learn more if you take CSCI3420 / CENG3420.

Summary

- C Compiler: There are different C compilers in the world, and a C compiler contains 4 components
- Preprocessor: Handle most lines start with #. Do copy and paste, search and replace, etc. Output a processed source file
- Compiler: Turn your code into assembly code, and optionally optimize your code
- Assembler: Translate the assembly code into machine code. The output is called object file
- Linker: Combine object files and libraries into an executable
- `int`: 4 bytes in 32-/64-bit environment. Usually use 2's complement. Results which out of the range occur overflow.
- `float` and `double`: Use 4 and 8 bytes respectively. Usually use IEEE 754 standard. Limited precision.